Argo CD & Argo Rollouts

Introduction to GitOps

Introduction to GitOps

- ✅ The Traditional Push Model & Its Drawbacks
- ✅ The GitOps Workflow
- ✅ The Four Principles of GitOps

Argo CD & Argo Rollouts

Introduction to GitOps

Getting Started with Argo CD

Getting Started with Argo CD

- ✅ Installing Argo CD via Helm
- ✅ Accessing the Argo CD Web UI
- ✅ The Argo CD Command Line Interface (CLI)

Introduction to GitOps

Getting Started with Argo CD

Core Argo CD Concepts

Core Argo CD Concepts

- ✅ **Argo CD Architecture & Components**
- ✅ **The Application Custom Resource (CRD)**
- ✅ **Deploying Your First Application**
- ✅ **The Sync Process & Application State**

Introduction to GitOps

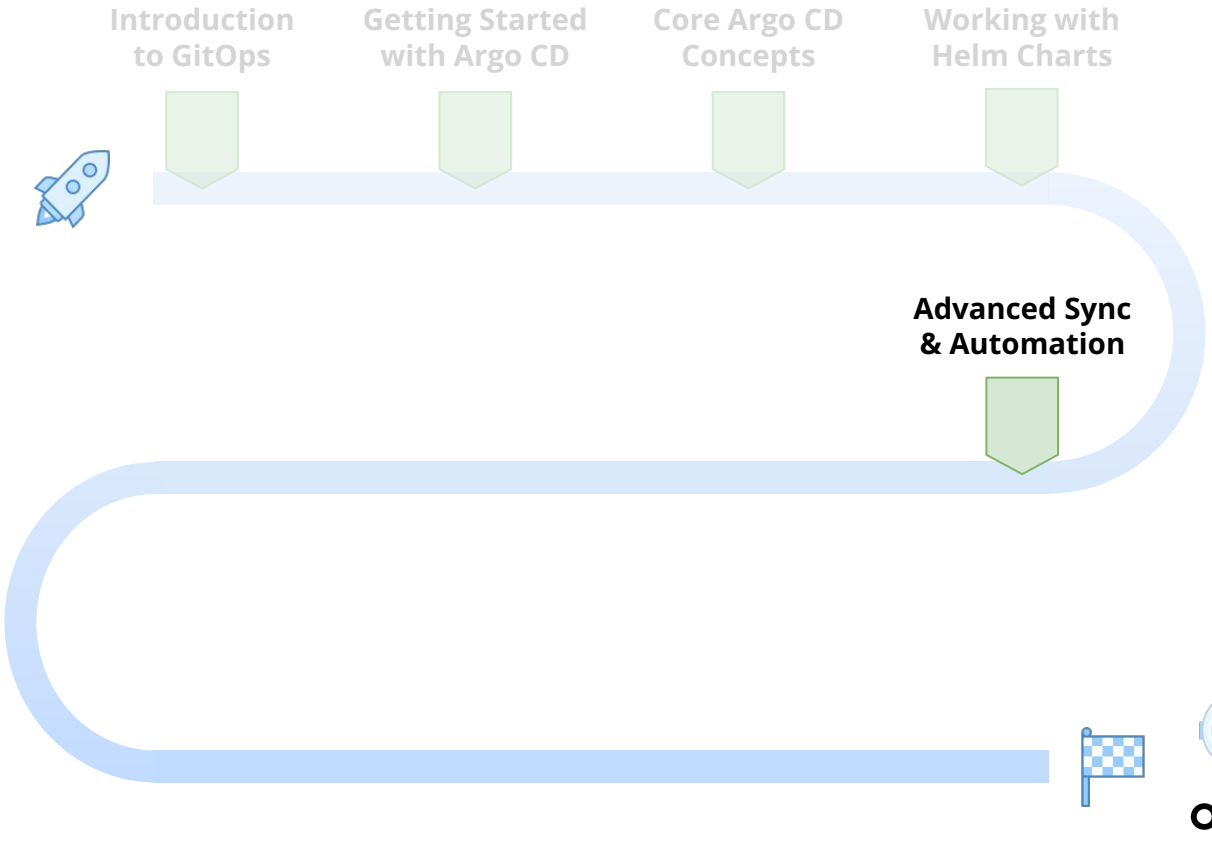
Getting Started with Argo CD

Core Argo CD Concepts

Working with Helm Charts

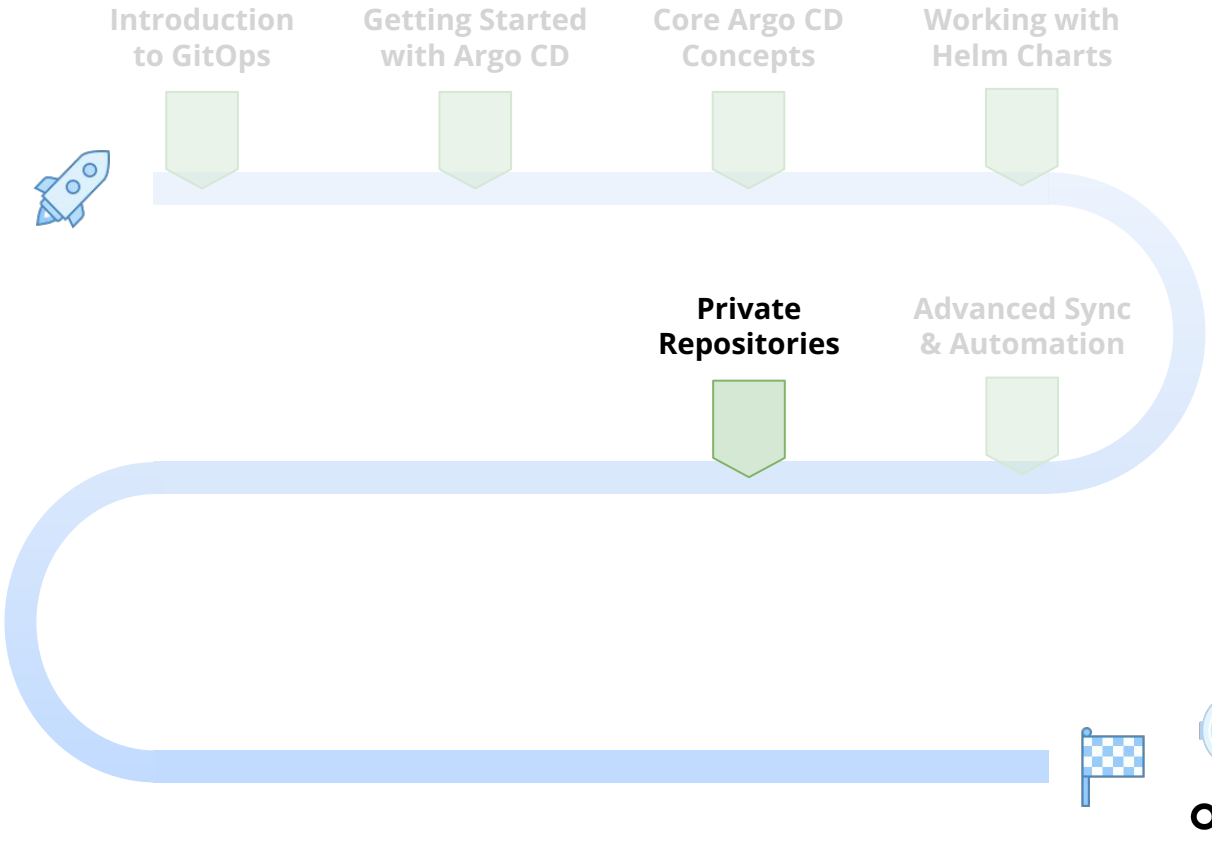
Working with Helm Charts

- ✅ **Helm Integration in Argo CD**
- ✅ **Deploying Public Helm Charts**
- ✅ **Customizing Helm Values & Precedence**



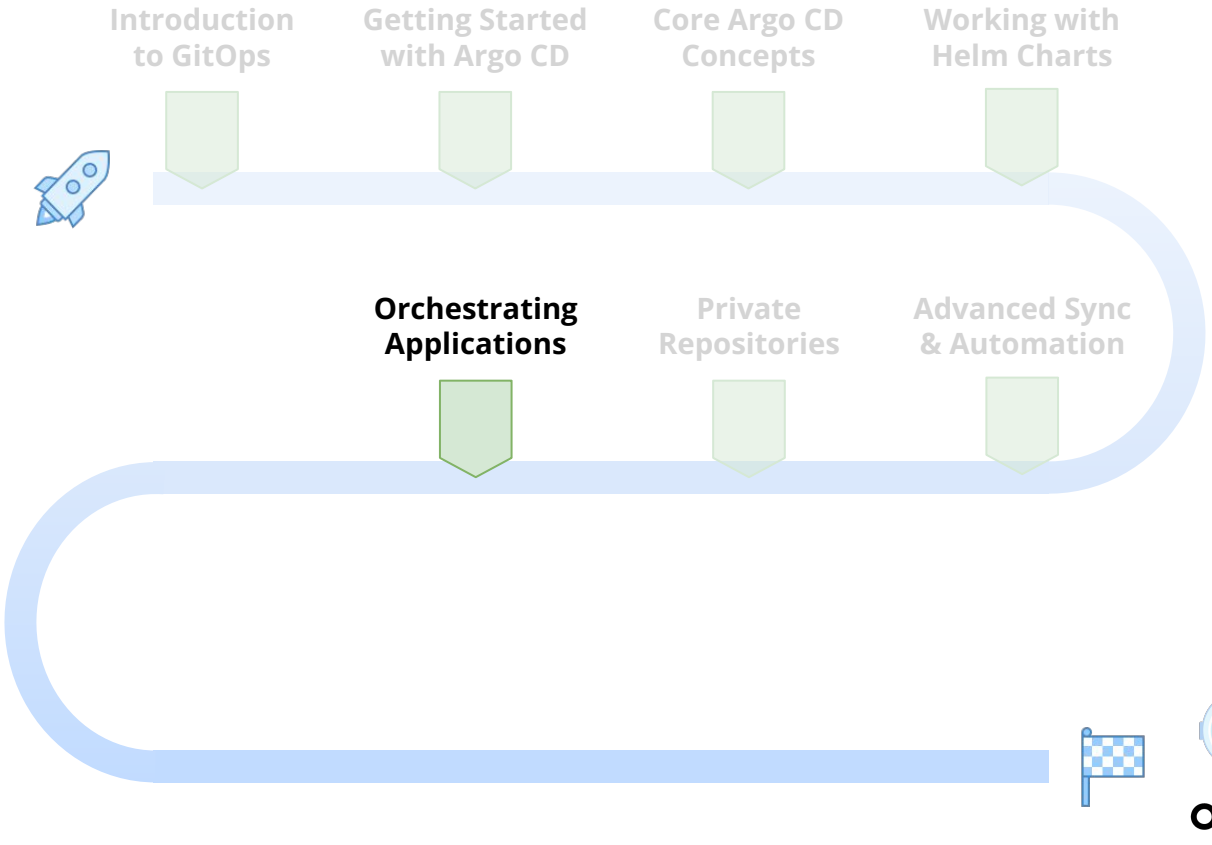
Advanced Sync & Automation

- ✅ Automated Syncing
- ✅ Pruning Orphaned Resources
- ✅ Self-Healing & Drift Correction



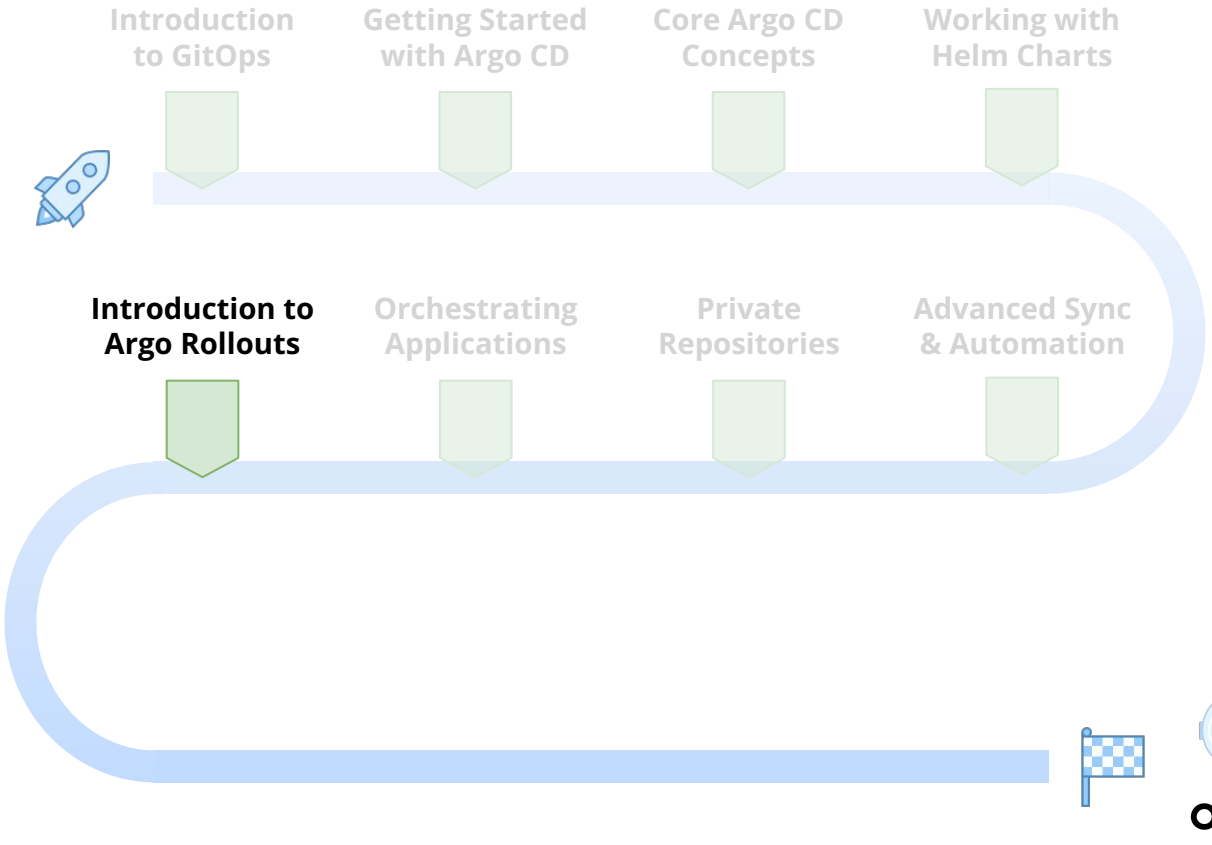
Private Repositories

- ✅ **Connecting to Private Git Repositories**
- ✅ **Authentication via HTTPS**
- ✅ **Authentication via SSH**



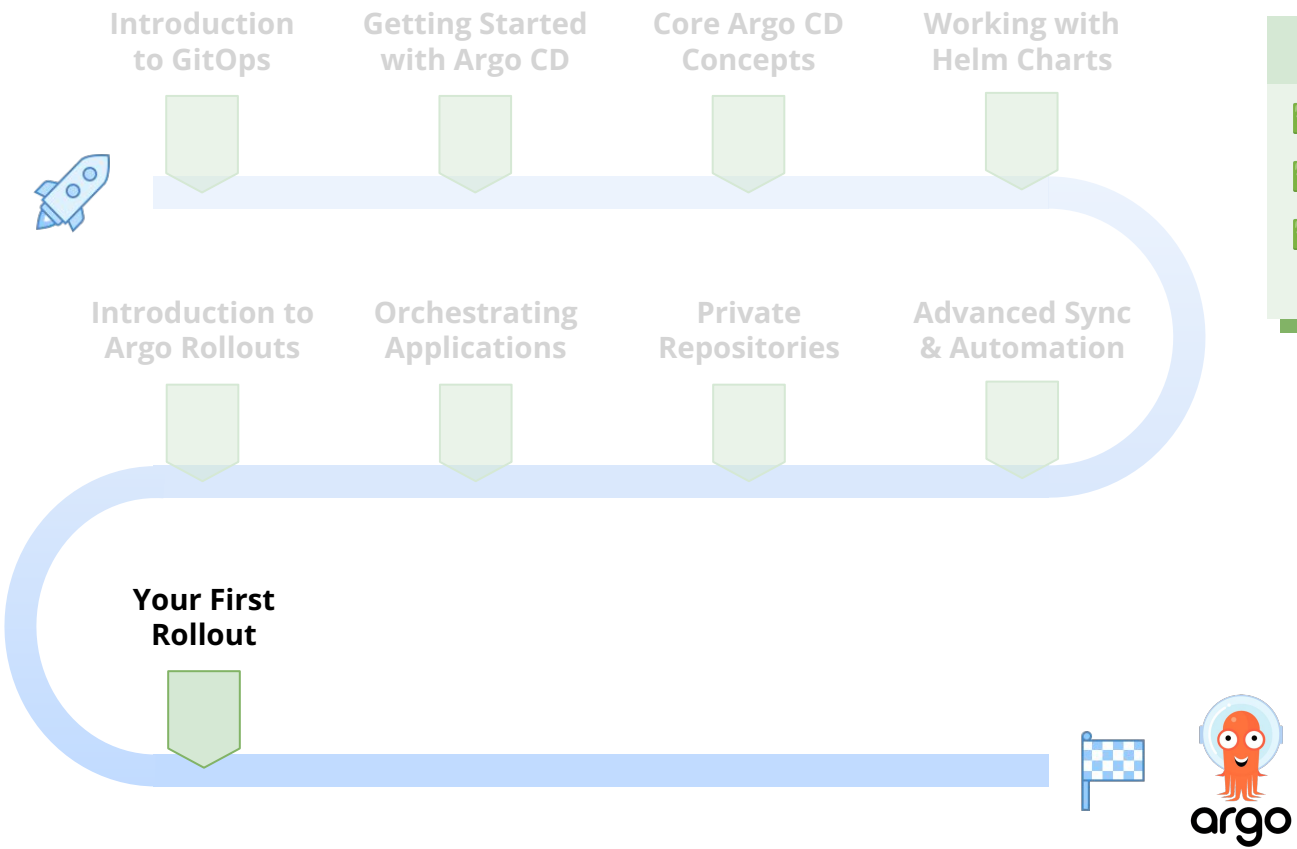
Orchestrating Applications

- ✅ Argo CD Projects & Multi-Tenancy
- ✅ Sync Phases & Hooks
- ✅ Sync Waves



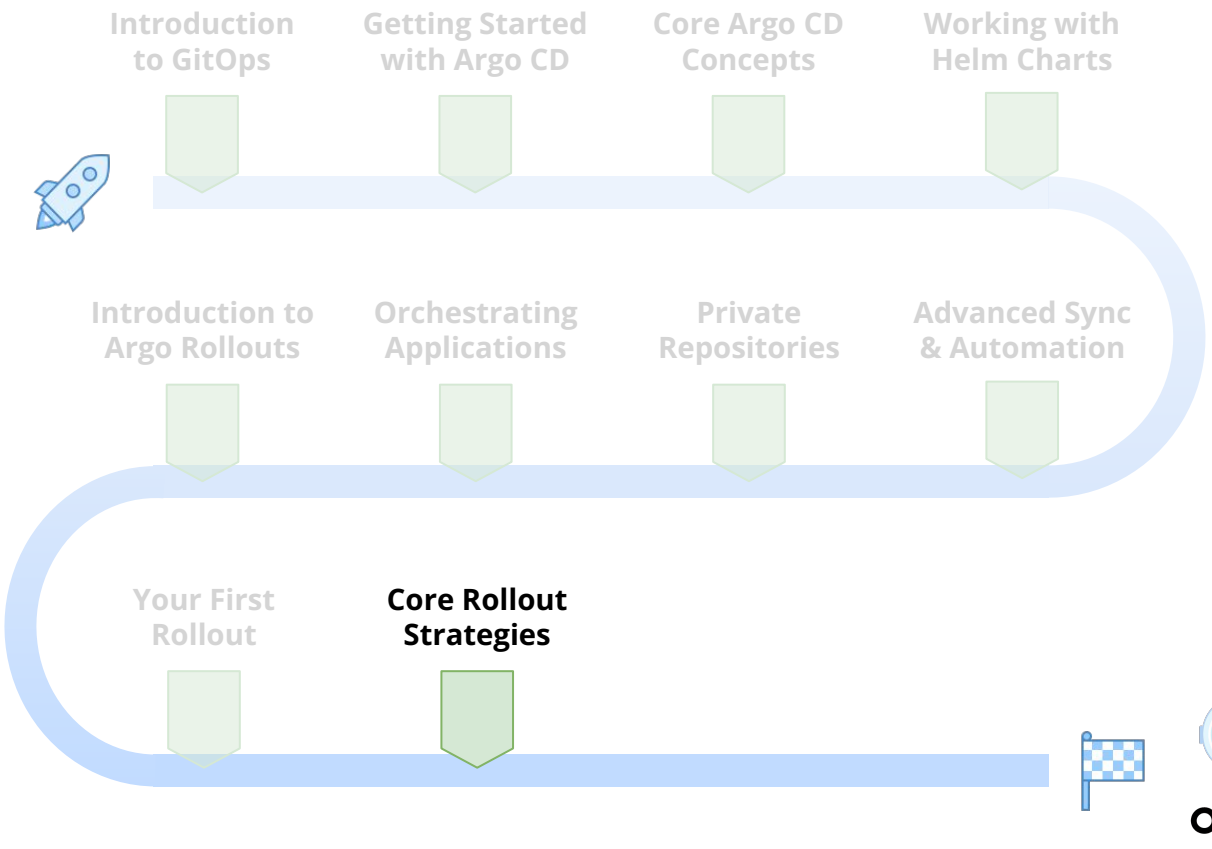
Introduction to Argo Rollouts

- ✅ Limitations of Standard Kubernetes Deployments
- ✅ Installing Argo Rollouts
- ✅ The Argo Rollouts Dashboard



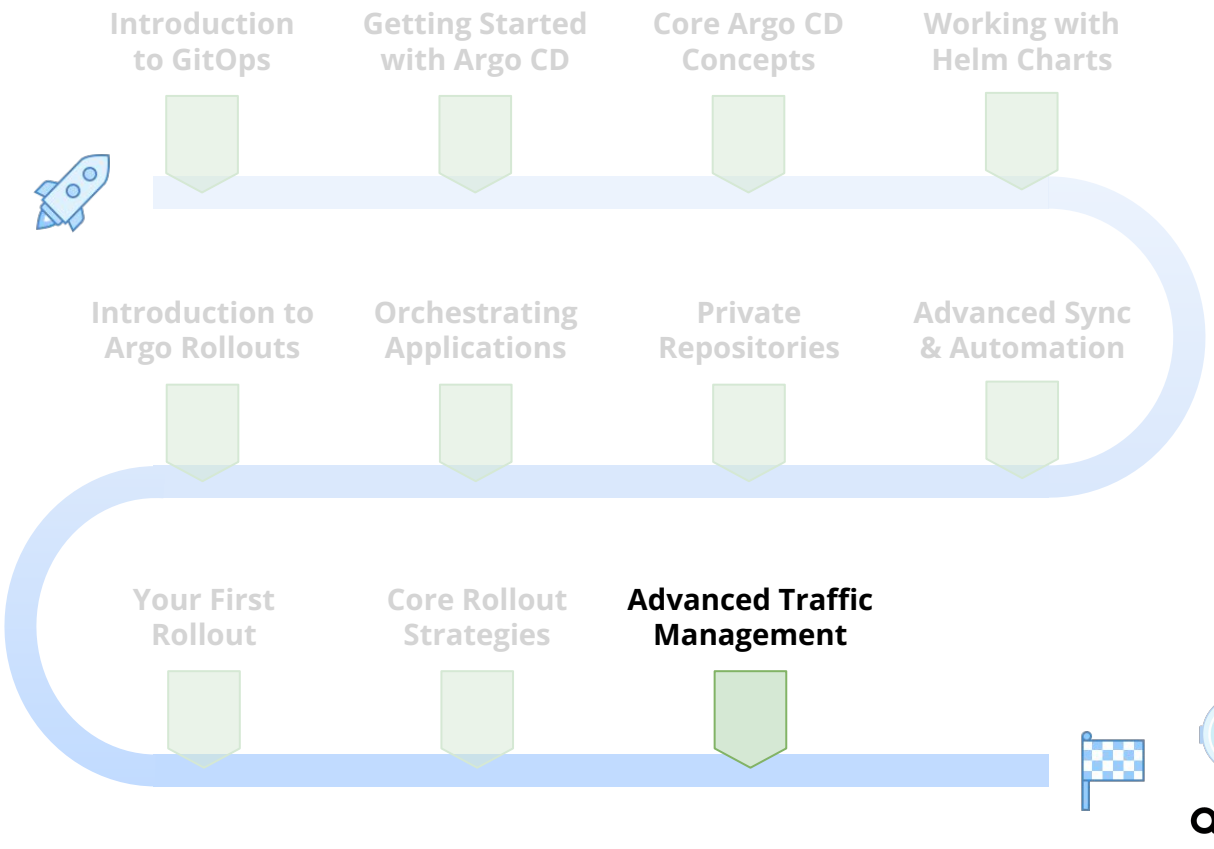
Your First Rollout

- ✅ The Rollout Custom Resource
- ✅ Creating Your First Rollout
- ✅ Managing Rollouts



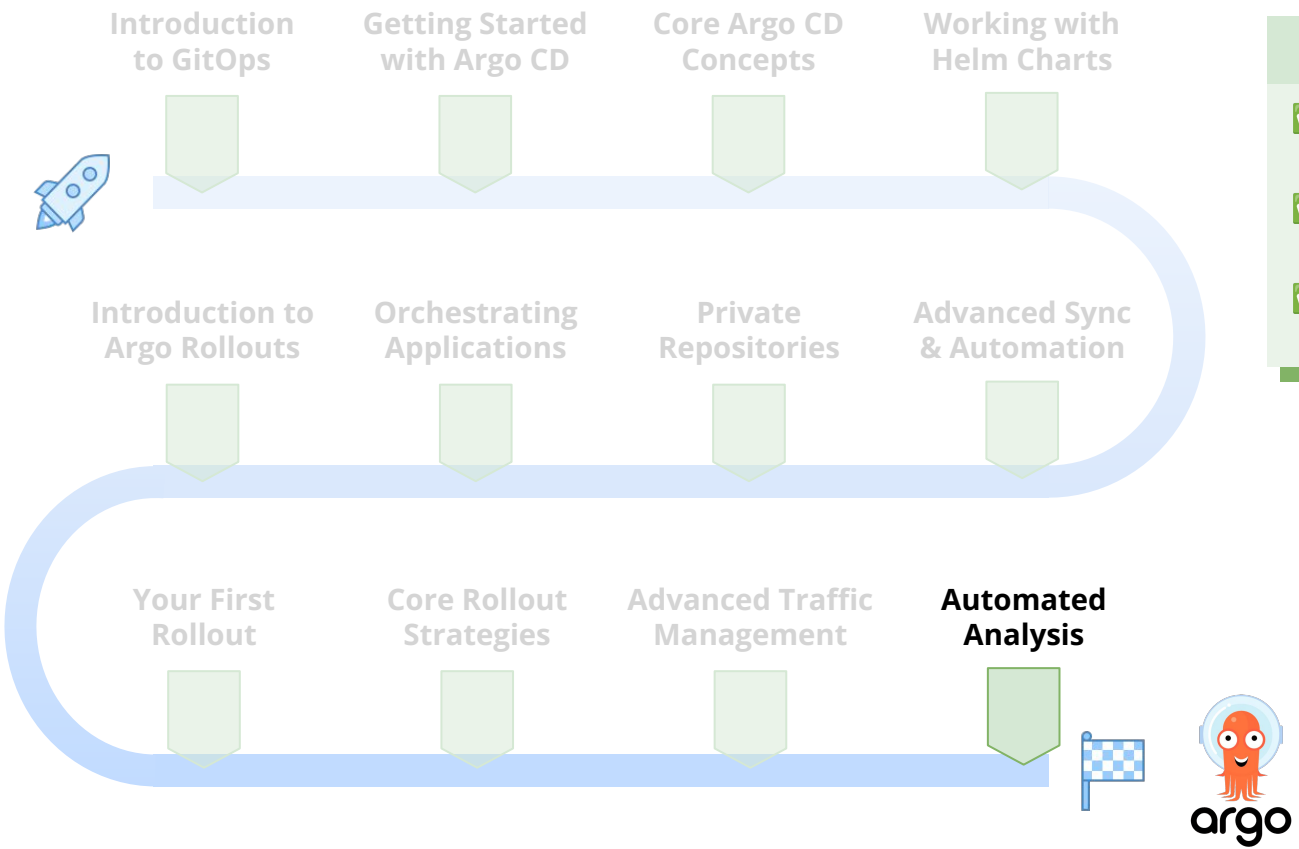
Core Rollout Strategies

- ✅ **Blue-Green Deployment Strategy**
- ✅ **Canary Deployment Strategy**
- ✅ **Further Canary Patterns**



Advanced Traffic Management

- ✅ **Limitations of Replica-Weighted Strategies**
- ✅ **Gateway API & Environment Setup**
- ✅ **Traffic-Weighted Canary Deployments**
- ✅ **Header-Based Routing**



Automated Analysis

- ✅ **Prometheus Architecture & Setup**
- ✅ **Automated Analysis & Promotion**
- ✅ **Analysis in Canary and Blue-Green Strategies**

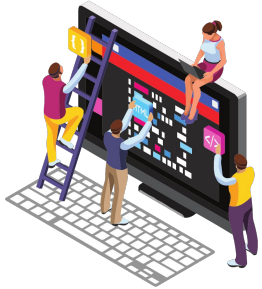
Introduction

Aligning Expectations

Aligning Expectations

What this course is about

Covering the most important aspects of working with Argo CD and Argo Rollouts in real-world projects



Gaining hands-on experience through practical labs and projects

Going beyond the basics and covering topics like Argo CD Hooks, Sync Phases and Waves, advanced Rollout Patterns, and much more!

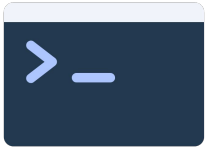
By the end of this course, you will have a strong **practical and theoretical foundation** of both **Argo CD** and **Argo Rollouts**, and you will be able to take this knowledge and apply it to **real-world, production projects**.

Aligning Expectations

Requirements and prerequisites for making the most of this course

Familiarity with programming and Git.

Familiarity with Docker, Kubernetes and Helm.



Familiarity with the terminal / command line.

Argo CD

Introduction to GitOps

Introduction to GitOps

Section overview

1 The Traditional Push Model & Its Drawbacks

1. Overview of the standard CI/CD deployment flow.
2. Explain its key challenges: poor auditability, stressful rollbacks, and inconsistent environments.

2 The GitOps Workflow

1. Moving from a "Push" model to a "Pull" model with Argo CD.

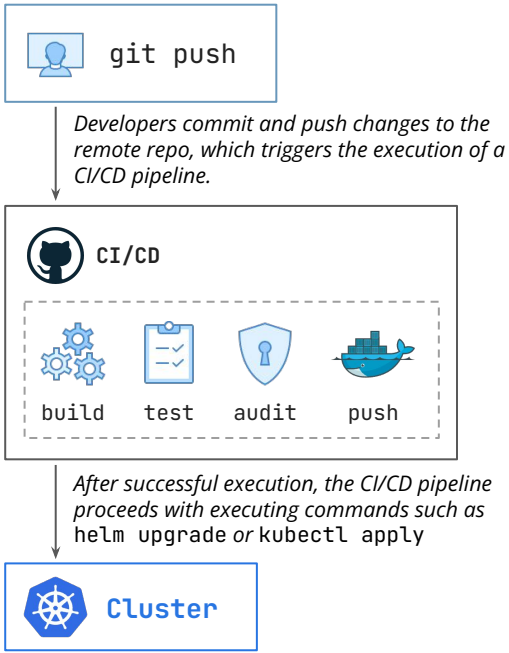
3 The Four Principles of GitOps

Argo CD

The GitOps Philosophy

The GitOps Philosophy

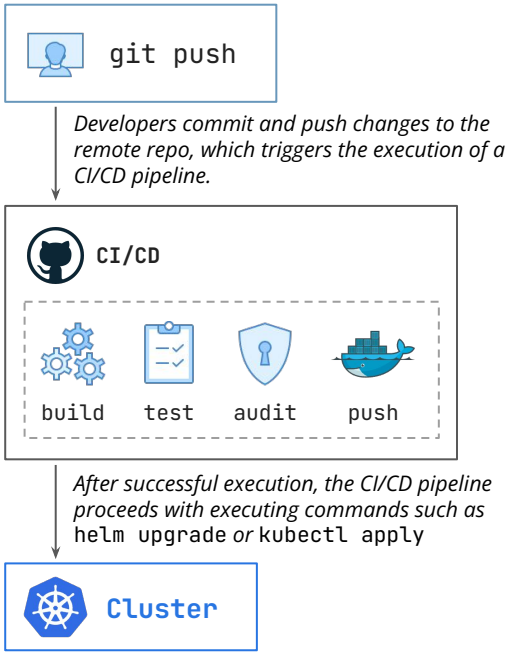
What are the disadvantages of the traditional deployment flow?



The GitOps Philosophy

What are the disadvantages of the traditional deployment flow?

Although widely adopted, the *push* model has several drawbacks:



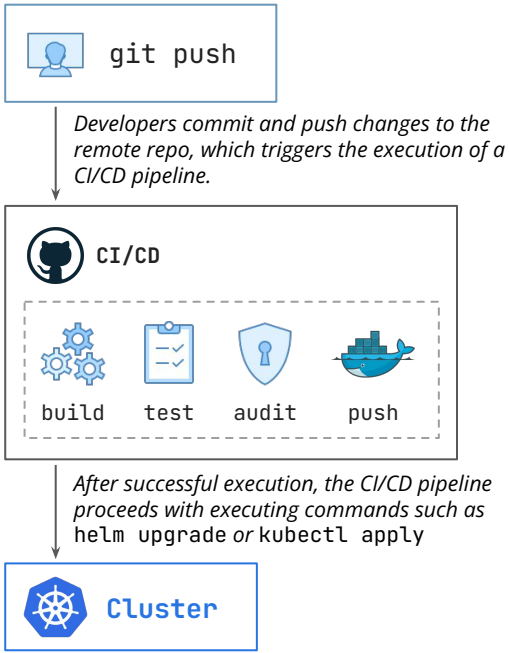
The GitOps Philosophy

What are the disadvantages of the traditional deployment flow?

Although widely adopted, the *push* model has several drawbacks:

Configuration Drift

> **What happens if a developer runs `kubectl scale deployment my-app --replicas=0` directly in the cluster?** The live state of the cluster is now different from the configuration stored in Git. Nobody knows what the real desired state is. Git says one thing, but the cluster is doing another.



The GitOps Philosophy

What are the disadvantages of the traditional deployment flow?

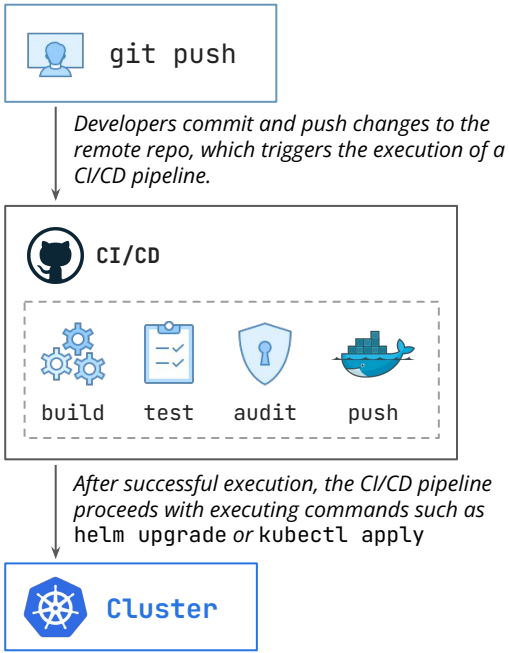
Although widely adopted, the *push* model has several drawbacks:

Configuration Drift

What happens if a developer runs `kubectl scale deployment my-app --replicas=0` directly in the cluster? The live state of the cluster is now different from the configuration stored in Git. Nobody knows what the real desired state is. Git says one thing, but the cluster is doing another.

Poor Auditability

How do you know who scaled the deployment and why? There's little to no audit trail. The "source of truth" is just the current state of the cluster, with no history.



The GitOps Philosophy

What are the disadvantages of the traditional deployment flow?

Although widely adopted, the *push* model has several drawbacks:

Configuration Drift

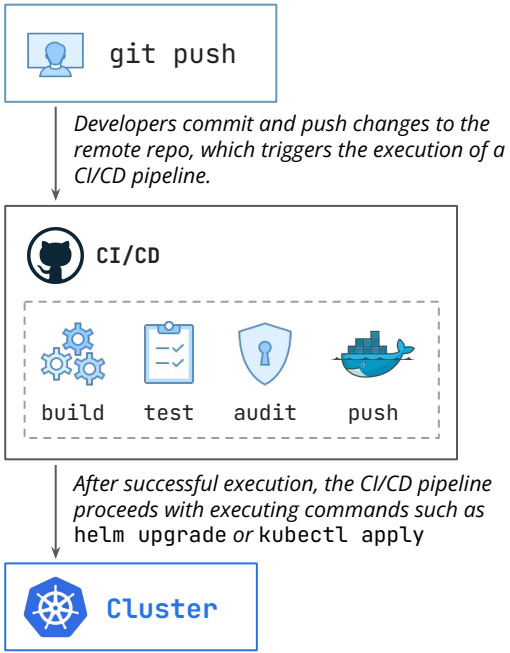
What happens if a developer runs `kubectl scale deployment my-app --replicas=0` directly in the cluster? The live state of the cluster is now different from the configuration stored in Git. Nobody knows what the real desired state is. Git says one thing, but the cluster is doing another.

Poor Auditability

How do you know who scaled the deployment and why? There's little to no audit trail. The "source of truth" is just the current state of the cluster, with no history.

Difficult Rollbacks

Reverting a bad deployment is often not straightforward, especially when multiple services are involved. It often requires a high-stress, manual process of finding the last good artifact and re-running a pipeline. This is slow and highly prone to human error during an outage.



The GitOps Philosophy

What are the disadvantages of the traditional deployment flow?

Although widely adopted, the *push* model has several drawbacks:

Configuration Drift

What happens if a developer runs `kubectl scale deployment my-app --replicas=0` directly in the cluster? The live state of the cluster is now different from the configuration stored in Git. Nobody knows what the real desired state is. Git says one thing, but the cluster is doing another.

Poor Auditability

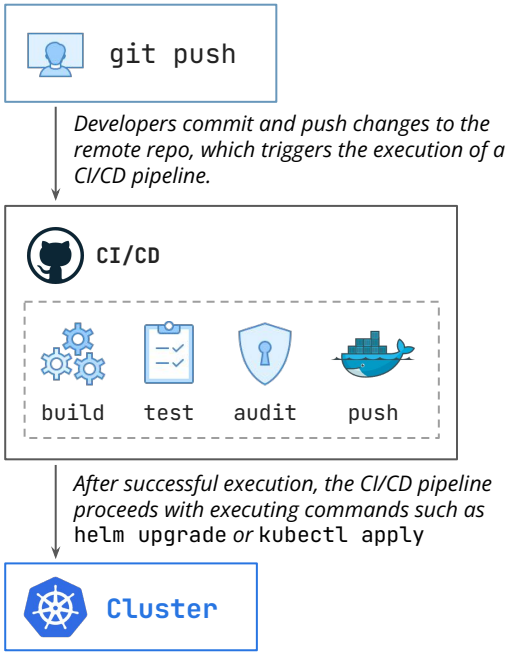
How do you know who scaled the deployment and why? There's little to no audit trail. The "source of truth" is just the current state of the cluster, with no history.

Difficult Rollbacks

Reverting a bad deployment is often not straightforward, especially when multiple services are involved. It often requires a high-stress, manual process of finding the last good artifact and re-running a pipeline. This is slow and highly prone to human error during an outage.

Inconsistent Deployments and Configuration

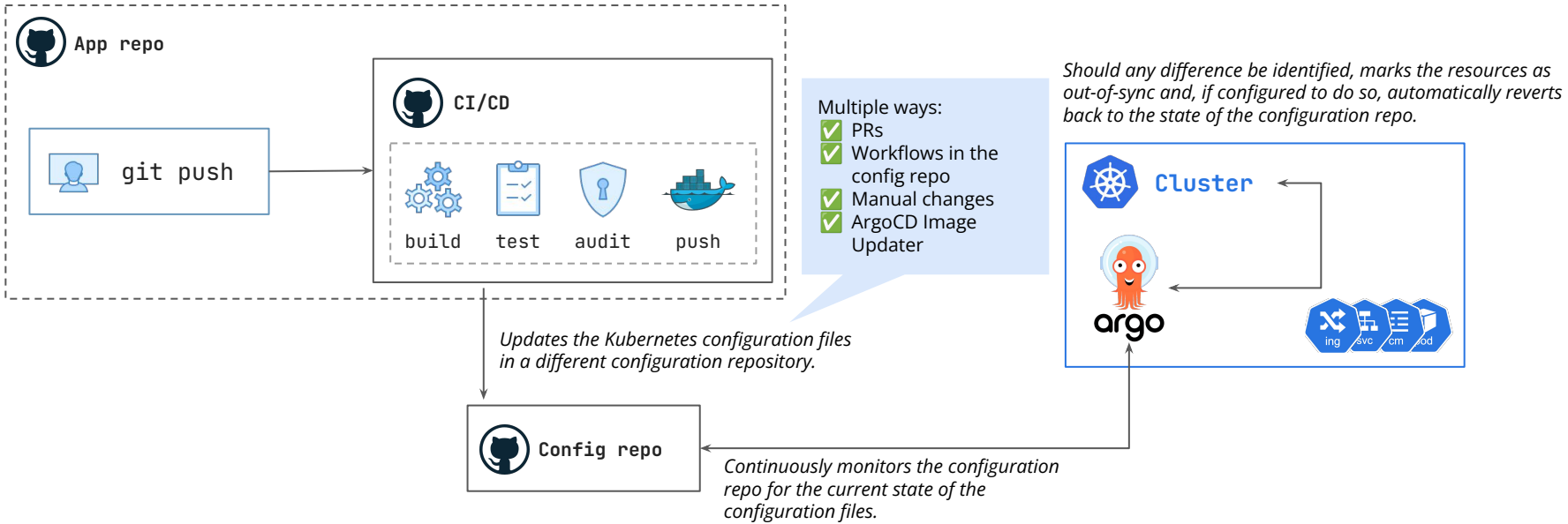
Since configuration might drift, it becomes increasingly harder to have a clear view of each environment configuration, also opening the door to bigger problems when promoting a certain release from a lower to a higher environment.



Argo CD

The GitOps Philosophy

How does the GitOps flow tackle these issues?



The GitOps Philosophy

GitOps Principles

Declarative

A system managed by GitOps must have its **desired state expressed declaratively**.

Versioned and Immutable

The desired state is stored in a way that enforces **immutability**, **versioning** and **retains a complete version history**.

Pulled Automatically

Software agents **automatically pull the desired state** declarations from the source.

Continuously Reconciled

Software agents **continuously observe the actual system state** and attempt to apply the desired state.

Argo CD

Getting Started with Argo CD

Getting Started with Argo CD

Section overview

1. Installing Argo CD via Helm
2. Accessing the Argo CD Web UI
3. The Argo CD Command Line Interface (CLI)
    1. Installing the `argocd` CLI tool on your local machine.
    2. Establishing an authenticated session with the local Argo CD server.

Argo CD

Core Argo CD Concepts

Core Argo CD Concepts

Section overview

1 Argo CD Architecture & Components

1. Understanding the specific roles of the API Server, Repository Server, and Application Controller.
2. How these components interact to generate manifests and monitor the cluster state.

2 The `Application` Custom Resource (CRD)

1. Understanding the schema: Source (Git), Destination (Cluster), and Sync Policies.
2. Distinguishing between the Argo CD `Application` resource and standard Kubernetes manifests.

3 Deploying Your First Application

1. Creating an Application manifest via YAML to deploy the "Guestbook" app.
2. Visualizing the deployment in the Argo CD UI and CLI.

Core Argo CD Concepts

Section overview

4 The Sync Process & Application State

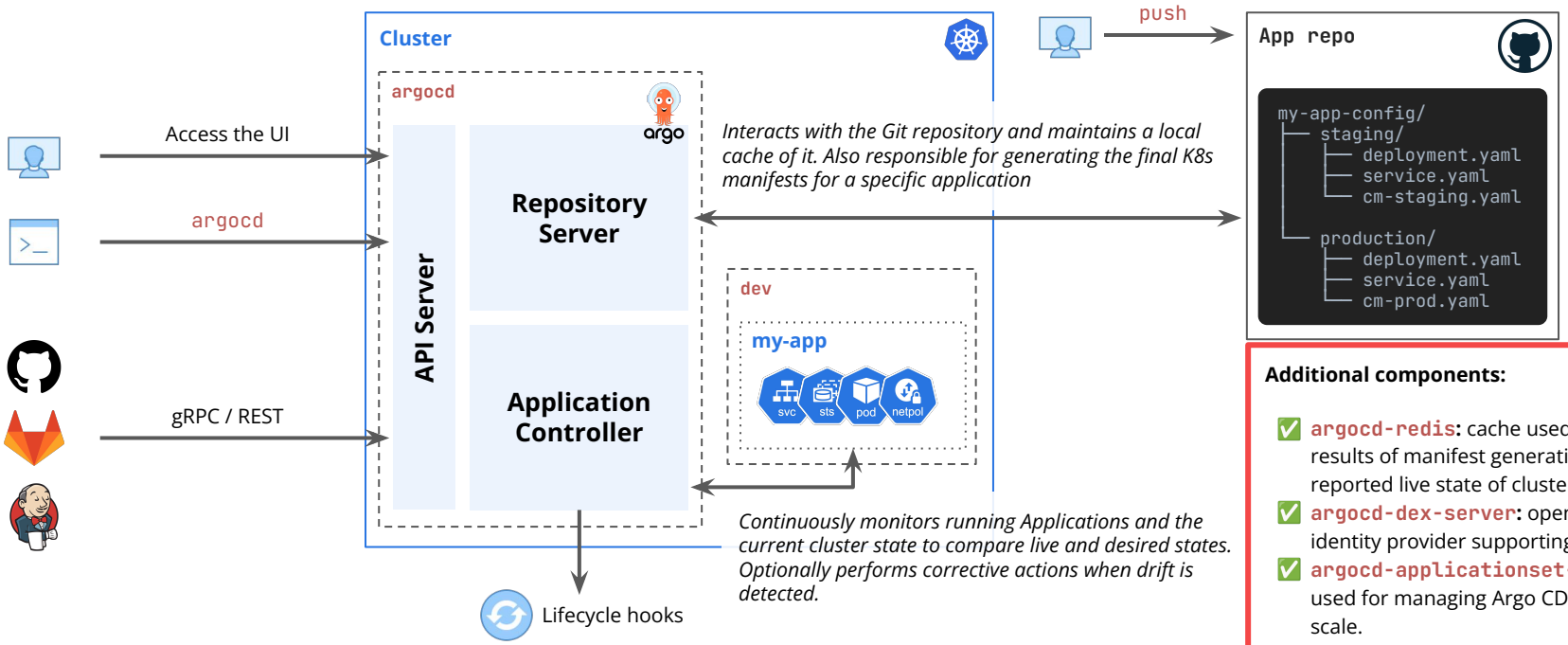
1. Understanding "Synced" vs. "Out of Sync" (Configuration Drift).
2. Differentiating between application configuration (Sync) and actual resource health (Healthy/Degraded).
3. Executing the full GitOps loop: modifying our source code, detecting changes, and synchronizing the cluster.

Argo CD

The Argo CD Components

The Argo CD Components

What do we get when we install Argo CD in a K8s cluster?



Additional components:

- ✅ **`argocd-redis`**: cache used to store the results of manifest generation and the reported live state of cluster resources.
- ✅ **`argocd-dex-server`**: open-source identity provider supporting OIDC & SAML.
- ✅ **`argocd-applicationset-controller`**: used for managing Argo CD applications at scale.

Argo CD

The Application CRD

The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

- The purpose of the Application CRD is to be a declarative contract that describes several important pieces of information for Argo CD to do its job:
  - **Project:** Under which Argo CD project should the application be managed?
  - **Source (`spec.source`):** *WHERE is the desired state defined?* This normally includes a Git repository URL, a branch, and a path or Helm chart).
  - **Destination (`spec.destination`):** *WHERE should the manifests from that source be deployed?* More specifically, which cluster and namespace?
  - **[Optional] Sync Policy (`spec.syncPolicy`):** *HOW should the deployment be managed?* Should it be automated, prune resources, self-heal, etc.?

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook-prod
  namespace: argocd
spec:
  project: default
  source:
    repoURL: <YOUR_REPO_URL>
    targetRevision: HEAD
    path: guestbook

  destination:
    server: <YOUR_K8S_SERVER>
    namespace: guestbook-prod
```


The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

Argo CD Applications vs. Kubernetes Manifests		
Concept	**Application CRD**	**Kubernetes Manifests**

The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

Argo CD Applications vs. Kubernetes Manifests		
Concept	**Application CRD**	**Kubernetes Manifests**
`kind:`	`Application`	`Deployment`, `Service`, `ConfigMap`, etc.

The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

Argo CD Applications vs. Kubernetes Manifests		
Concept	**Application CRD**	**Kubernetes Manifests**
`kind:`	`Application`	`Deployment`, `Service`, `ConfigMap`, etc.
Purpose	Declarative definition of a **contract** for Argo CD to manage a specific set of Kubernetes manifests.	The **definition** of the multiple components and resources of your running application.

The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

Argo CD Applications vs. Kubernetes Manifests		
Concept	**Application CRD**	**Kubernetes Manifests**
`kind:`	`Application`	`Deployment`, `Service`, `ConfigMap`, etc.
Purpose	Declarative definition of a **contract** for Argo CD to manage a specific set of Kubernetes manifests.	The **definition** of the multiple components and resources of your running application.
Information	**Where** is the code? **Where** should it be deployed? **How** should it be synced?	**Which** container image to run? **Which** ports to open? **How many** replicas to create?

The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

Argo CD Applications vs. Kubernetes Manifests		
Concept	**Application CRD**	**Kubernetes Manifests**
`kind:`	`Application`	`Deployment`, `Service`, `ConfigMap`, etc.
Purpose	Declarative definition of a **contract** for Argo CD to manage a specific set of Kubernetes manifests.	The **definition** of the multiple components and resources of your running application.
Information	**Where** is the code? **Where** should it be deployed? **How** should it be synced?	**Which** container image to run? **Which** ports to open? **How many** replicas to create?
Where in Cluster	The `argocd` namespace.	The target application namespace (for example, `guestbook-staging`).

The `Application` CRD

Understand the role of the Application CRD and how Argo CD uses it

Argo CD Applications vs. Kubernetes Manifests		
Concept	**Application CRD**	**Kubernetes Manifests**
`kind:`	`Application`	`Deployment`, `Service`, `ConfigMap`, etc.
Purpose	Declarative definition of a **contract** for Argo CD to manage a specific set of Kubernetes manifests.	The **definition** of the multiple components and resources of your running application.
Information	**Where** is the code? **Where** should it be deployed? **How** should it be synced?	**Which** container image to run? **Which** ports to open? **How many** replicas to create?
Where in Cluster	The `argocd` namespace.	The target application namespace (for example, `guestbook-staging`).
Who Cares About It?	The **Argo CD controllers**.	The **Kubernetes controllers** (like the deployment controller).

Argo CD

Sync and Health Status

Sync and Health Statuses

Explore the different sync phases and health statuses

- **Sync status** refers to whether the live state, or the deployed kubernetes resources, match the desired state, or the configuration files in the application repository.

`Synced`	The Live State **matches** the Desired State
`OutOfSync`	The Live State **does not match** the Desired State. **Configuration drift** has happened.
`Progressing`	The `Application` is currently undergoing a sync operation. Is temporary.

- **Health status** refers to whether the live state, or the deployed kubernetes resources, is in a healthy status. Argo CD has built-in health checks for different Kubernetes resources.

`Healthy`	All the resources associated with the application are in a good state.
`Degraded`	At least one resource is in a failed or unhealthy state. The application might be perfectly `Synced`, but still be `Degraded`.

Argo CD

Working with Helm Charts

Working with Helm Charts

Section overview

1 Helm Integration in Argo CD

1. Argo CD as a Helm template engine.
2. Understanding the workflow from template generation through `kubectl apply`.

2 Deploying Public Helm Charts

1. Configuring Argo CD Applications to source from public Helm repositories.
2. Managing external charts just like internal code.

3 Customizing Helm Values & Precedence

1. Exploring methods for overriding values and their precedence.

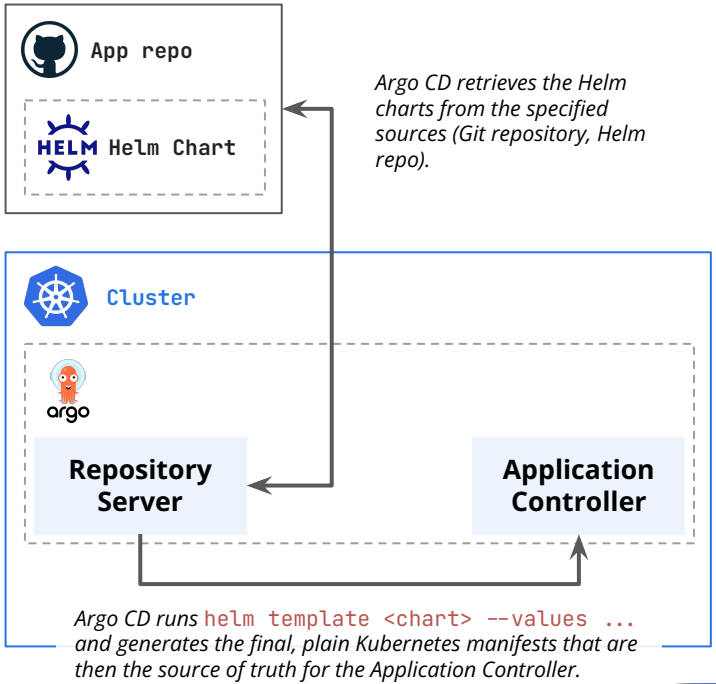
Argo CD

Deploying Helm Charts

Deploying Helm Charts

Understand how Argo CD uses and deploys Helm Charts

- Argo CD is a `helm template` engine.
  - This is the single most important concept to understand. When you manage a Helm chart with Argo CD, **it does not run `helm install` or `helm upgrade`**.
- As such, there is no Helm history.
- There is also no `helm install`, `helm upgrade`, or `helm uninstall`.
- You can use Argo CD to deploy both charts stored in your own repositories, as well as charts that are publicly available.



Argo CD

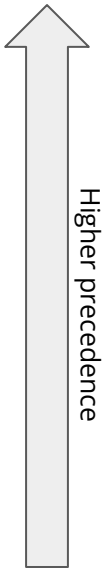
Overriding Chart Values

Overriding Chart Values

Explore the multiple ways to override Chart values and their precedence

```yaml
spec:
 source:
   repoURL: https://kubernetes.github.io/dashboard/
   chart: kubernetes-dashboard
   targetRevision: 7.13.0
   helm:
     valueFiles:
       - values.yaml
       - custom-values.yaml
     valuesObject:
       kong:
         enabled: true
     parameters:
       - name: 'nginx.enabled'
         value: 'false'
```

4. Entries specified under the `parameters` option
   - Later entries in the list take precedence over earlier entries.
3. Entries specified under the `valuesObject` option
2. Files specified under the `valueFiles` option
   - Later entries in the list take precedence over earlier entries.
1. The Chart's own `values.yaml` file



Argo CD

Advanced Sync and Automation

Advanced Sync and Automation

Section overview

1 Automated Syncing

1. Moving beyond manual approval to fully automated deployments.
2. Understanding the safety mechanisms and when to enable automation.

2 Pruning Orphaned Resources

1. Handling resources that are deleted from Git but remain in the cluster.
2. Enabling the `prune` option to automatically clean up and remove legacy resources during sync.

3 Self-Healing & Drift Correction

1. Closing the loop on configuration drift caused by manual cluster changes.
2. How the self-heal policy immediately detects and reverts unauthorized changes to match the desired state.

Argo CD

Automation, Pruning, Self-Healing

Automation, Pruning, Self-Healing

Understand how to achieve a fully automated GitOps workflow

- **Automated Sync:** By default, Argo CD only reports `OutOfSync`. It does not automatically update the Kubernetes resources.
  - **Safety feature:** It gives you a chance to review the planned changes before clicking `SYNC`.
  - **Automated Sync** is the policy that allows ArgoCD to automatically apply the changes without manual approval.
- **Pruning:** By default, Argo CD does not delete resources if it cannot find their manifests in the source. In other words, if you delete a manifest, by default Argo CD **will not** delete that resource in the cluster.
  - **Pruning** allows Argo CD to actually clean up orphaned resources from the cluster if their manifests cannot be found.
- **Self-Healing:** Automated sync policies only trigger when the *Git repository* changes, not if someone changes something *in the cluster*.
  - **Self-healing** is the policy that closes this final gap. When you enable self-healing, Argo CD will immediately revert changes to the cluster resources that lead to resources being in a different state than that specified in the Git repository.

Argo CD

Private Repositories

Private Repositories

Section overview

1 Connecting to Private Git Repositories

1. Overview of the mechanism: Creating Kubernetes Secrets with specific labels and data schemas.
2. The two primary authentication methods: HTTPS (username + PAT) and SSH (private key).

2 Authentication via HTTPS

1. Generating Fine-Grained Personal Access Tokens in GitHub.
2. Configuring repository credentials via the Argo CD UI and the CLI.

3 Authentication via SSH

1. Generating SSH key pairs (ssh-keygen) and configuring Deploy Keys in the Git provider.
2. Creating the necessary Kubernetes Secrets for SSH authentication.

Argo CD

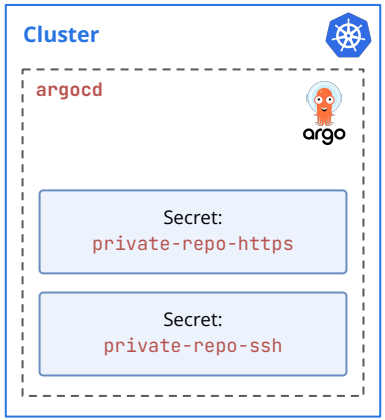
Private Repositories

Private Repositories

Understand how Argo CD connects to private repositories via HTTPS and SSH

app.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook-private
  namespace: argocd
spec:
  project: default
  source:
    repoURL: <YOUR_PRIVATE_REPO_URL>
    path: helm-guestbook
    targetRevision: HEAD
  destination:
    server: https://kubernetes.default.svc
    namespace: default
```



Argo CD will look for secrets within the `argocd` namespace for the provided repository. Once it finds a matching credential, it will use it to authenticate into the repository.

secret-https.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: private-repo-https
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
type: Opaque
stringData:
  type: "git"
  url: <YOUR_PRIVATE_REPO_URL>
  username: <YOUR_GIT_USERNAME>
  password: <YOUR_PAT>
```

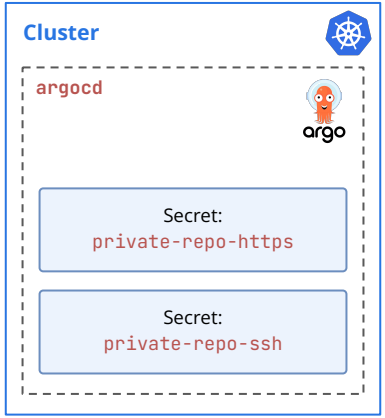
`argocd.argoproj.io/secret-type: repository` *is a mandatory label that defines which secrets contain repository credentials, and we must add it for Argo CD to find the correct credentials.*

Private Repositories

Understand how Argo CD connects to private repositories via HTTPS and SSH

app.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook-private
  namespace: argocd
spec:
  project: default
  source:
    repoURL: <YOUR_PRIVATE_REPO_URL>
    path: helm-guestbook
    targetRevision: HEAD
  destination:
    server: https://kubernetes.default.svc
    namespace: default
```



Argo CD will look for secrets within the `argocd` namespace for the provided repository. Once it finds a matching credential, it will use it to authenticate into the repository.

secret-https.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: private-repo-https
  namespace: argocd
  labels:
    argocd.argoproj
type: Opaque
stringData:
  type: "git"
  <YOUR_PRIVAT
  password: <YOUR_P
```

`argocd.argopro`
is a mandatory lab
repository credenti
find the correct cre

secret-ssh.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: private-repo-ssh
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
type: Opaque
stringData:
  type: "git"
  url: <YOUR_PRIVATE_REPO_URL>
  sshPrivateKey: |
    -----BEGIN OPENSSH PRIVATE KEY-----
    YOUR-SSH-PRIVATE-KEY-CONTENT-HERE
    -----END OPENSSH PRIVATE KEY-----
```

Argo CD

Orchestrating Applications

Orchestrating Applications

Section overview

1 Argo CD Projects & Multi-Tenancy

1. Logical grouping of applications for better organization and security.
2. Implementing guardrails: Restricting source repositories, destination clusters, and allowed resource kinds.

2 Sync Phases & Hooks

1. Running custom logic during the deployment lifecycle.
2. Configuring hook deletion policies (`BeforeHookCreation`, `HookSucceeded`) for resource cleanup.

3 Sync Waves

1. Defining precise deployment order using the `argocd.argoproj.io/sync-wave` annotation.
2. Combining waves with phases to orchestrate complex dependencies.

Argo CD

Projects

Projects

Manage and segregate multiple Argo CD applications at scale

What is a `Project`?

A `Project` is a logical grouping object that **exists only inside Argo CD**. This is not a cluster-wide resource like `Namespace`s.

Projects enable / support:

Organization & Multi-Tenancy: Provide a simple and effective way to group related applications (team, business unit).

Access Control (RBAC): Projects are the central point for configuring Role-Based Access Control (RBAC) in Argo CD, allowing you to set granular permissions for users.

Security Guardrails: Define guardrails for which source repositories, destinations, and Kubernetes resources applications belonging to the project can access.

project.yaml

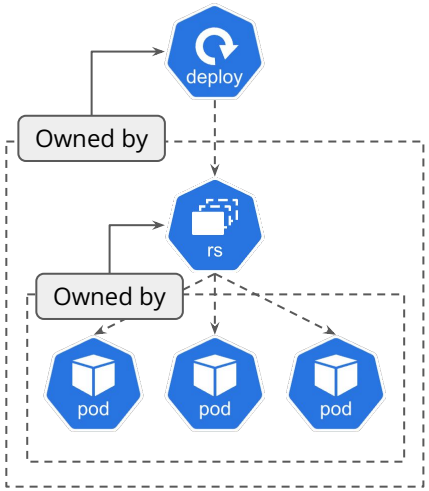
```yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: team-fin
  namespace: argocd
spec:
  description: "Project for Finance team"
  sourceRepos:
  - "<REPO_URL_1>"
  - "!<REPO_URL_2>"

  destinations:
  - namespace: finance
    server: 'https://kubernetes.default.svc'

  clusterResourceWhitelist:
  - group: ''
    kind: 'Namespace'
```

Argo CD

Propagation Policies

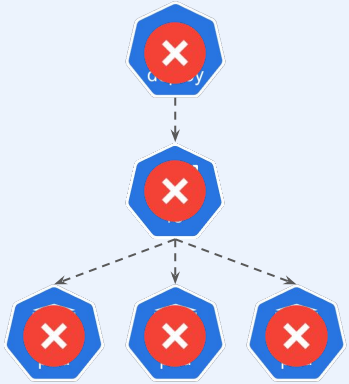
Propagation Policies

Understand how Kubernetes manages the resource deletion order



Foreground

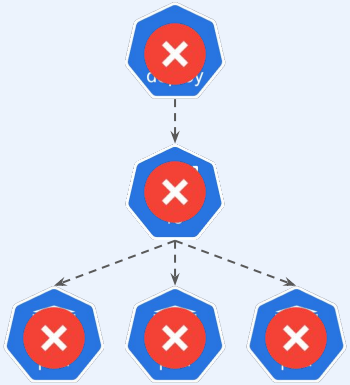
The owner object enters a "deletion in progress" state. The garbage collector deletes all dependents first, then deletes the owner.



Background

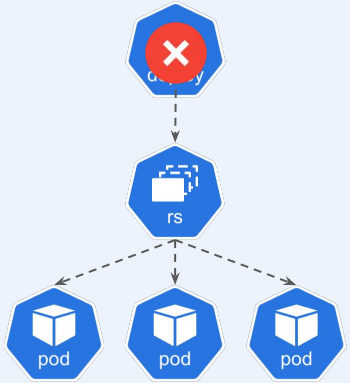
Default

The owner object is deleted immediately. The garbage collector then cleans up the dependent objects in the background.



Orphan

The owner object is deleted, but all of its dependent objects are left behind. They become "orphaned".

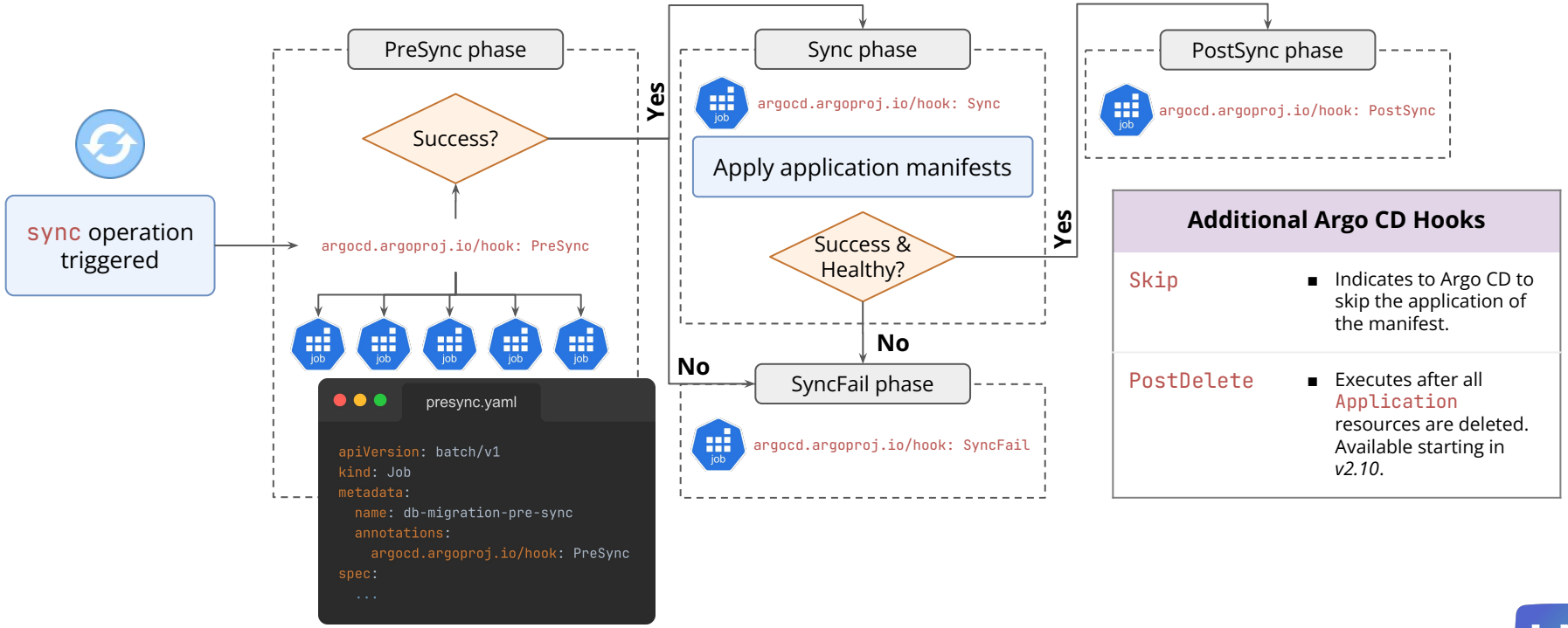


Argo CD

Sync Phases & Hooks

Sync Phases & Hooks

Understand how to run custom code during an Argo CD sync operation



Argo CD

Sync Waves

Sync Waves

Define the execution order of Kubernetes resources within Sync Phases

wave1.yaml

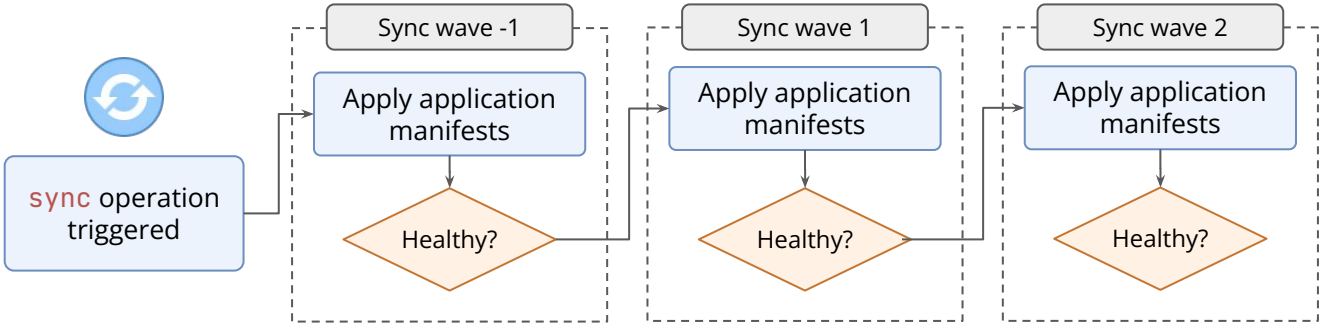
```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-db
  annotations:
    argocd.argoproj.io/sync-wave: "1"
```

wave2.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-application
  annotations:
    argocd.argoproj.io/sync-wave: "2"
```

wave_neg.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: my-ns
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
```

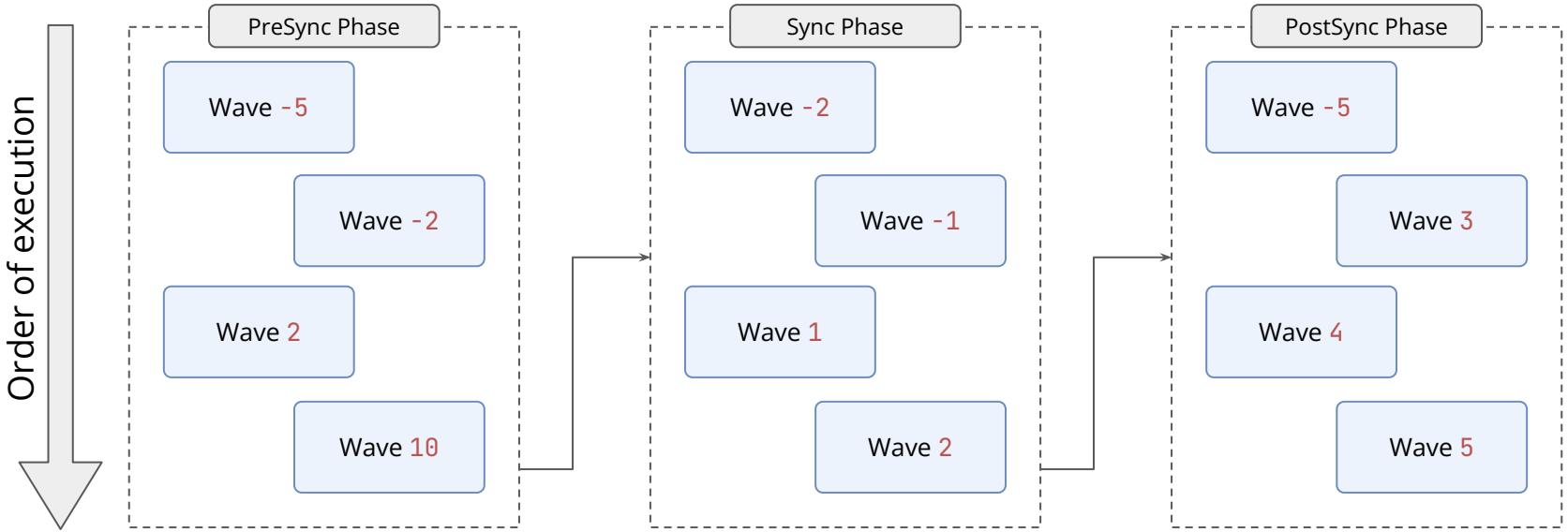


Sync waves and phases (hooks) can be combined!

When specifying both options, Argo CD will apply the manifests and execute the operations following the Sync Wave order within each Sync Phase!

Sync Waves

Define the execution order of Kubernetes resources within Sync Phases



Source: Argo CD Documentation

Argo Rollouts

Introduction to Argo Rollouts

Introduction to Argo Rollouts

Section overview

1. Limitations of Standard Kubernetes Deployments
2. Installing Argo Rollouts
    1. Deploying the Argo Rollouts controller via the official Helm chart.
    2. Installing the `kubectl argo rollouts` CLI plugin for local management.
3. The Argo Rollouts Dashboard
    1. Enabling the dashboard component via Helm values.
    2. Accessing the graphical interface, visualizing rollout progress, and managing specific releases via the UI.

Argo Rollouts

Limitations of Deployments

Limitations of Deployments

Why are `Deployments` not suitable for production workloads?

Traditional Kubernetes `Deployments` have several drawbacks when being updated:

Limitations of Deployments

Why are `Deployments` not suitable for production workloads?

Traditional Kubernetes `Deployments` have several drawbacks when being updated:

Too Fast and "All or Nothing"

Once a `RollingUpdate` begins, it proceeds to completion as quickly as it can create new pods. **There is no concept of pausing the rollout to observe the new version's behavior.** If you deploy a version with a bug, the `RollingUpdate` will happily continue replacing all of your stable pods with broken ones, rapidly escalating a small problem into a full-blown outage.

Limitations of Deployments

Why are `Deployments` not suitable for production workloads?

Traditional Kubernetes `Deployments` have several drawbacks when being updated:

Too Fast and "All or Nothing"

Once a `RollingUpdate` begins, it proceeds to completion as quickly as it can create new pods. **There is no concept of pausing the rollout to observe the new version's behavior.** If you deploy a version with a bug, the `RollingUpdate` will happily continue replacing all of your stable pods with broken ones, rapidly escalating a small problem into a full-blown outage.

"Success" is Defined Poorly

A readiness probe only confirms that a pod is ready to accept traffic. It does not confirm that the application is logically correct. As long as the readiness probe endpoint returns a `200 OK`, Kubernetes considers the pod "Ready" and continues the rollout.

Limitations of Deployments

Why are `Deployments` not suitable for production workloads?

Traditional Kubernetes `Deployments` have several drawbacks when being updated:

Too Fast and "All or Nothing"

Once a `RollingUpdate` begins, it proceeds to completion as quickly as it can create new pods. **There is no concept of pausing the rollout to observe the new version's behavior.** If you deploy a version with a bug, the `RollingUpdate` will happily continue replacing all of your stable pods with broken ones, rapidly escalating a small problem into a full-blown outage.

"Success" is Defined Poorly

A readiness probe only confirms that a pod is ready to accept traffic. It does not confirm that the application is logically correct. As long as the readiness probe endpoint returns a `200 OK`, Kubernetes considers the pod "Ready" and continues the rollout.

There is No Real Traffic Control

The `RollingUpdate` strategy ties instance scaling directly to traffic shifting. As soon as a new pod becomes "Ready", it's added to the Service's load balancing pool and begins receiving a portion of live user traffic.

Limitations of Deployments

Why are `Deployments` not suitable for production workloads?

Traditional Kubernetes `Deployments` have several drawbacks when being updated:

Too Fast and "All or Nothing"

Once a `RollingUpdate` begins, it proceeds to completion as quickly as it can create new pods. **There is no concept of pausing the rollout to observe the new version's behavior.** If you deploy a version with a bug, the `RollingUpdate` will happily continue replacing all of your stable pods with broken ones, rapidly escalating a small problem into a full-blown outage.

"Success" is Defined Poorly

A readiness probe only confirms that a pod is ready to accept traffic. It does not confirm that the application is logically correct. As long as the readiness probe endpoint returns a `200 OK`, Kubernetes considers the pod "Ready" and continues the rollout.

There is No Real Traffic Control

The `RollingUpdate` strategy ties instance scaling directly to traffic shifting. As soon as a new pod becomes "Ready", it's added to the Service's load balancing pool and begins receiving a portion of live user traffic.

Rollbacks Are Just Another Risky `RollingUpdate`

When you trigger a rollback on a `Deployment`, Kubernetes doesn't instantly switch back to the old version. Instead, it simply starts another `RollingUpdate` in reverse, replacing the bad pods with the last known-good version.

Argo Rollouts

Argo Rollouts

Your First Rollout

Your First Rollout

Section overview

1 The `Rollout` Custom Resource (CRD)

1. Transitioning from Deployment to Rollout.
2. Introducing the Canary and Blue-Green deployment strategies.

2 Creating Your First Rollout

1. Converting a standard `Deployment` manifest into a `Rollout` manifest.
2. Implementing a basic Canary strategy and verifying progress in the UI and CLI.

3 Managing Rollouts

1. Executing updates, visualizing progress and promoting rollouts.

Argo Rollouts

The `Rollout` CRD

The `Rollout` CRD

Understand the core resource in Argo Rollouts

deploy.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: guestbook-ui
spec:
  replicas: 5
  selector:
    matchLabels:
      app: guestbook
  template:
    metadata:
      labels:
        app: guestbook
    spec:
      containers:
      - name: ui
        image: guestbook:v1
```

rollout.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: guestbook-ui
spec:
  replicas: 5
  selector:
    matchLabels:
      app: guestbook
  template:
    metadata:
      labels:
        app: guestbook
    spec:
      containers:
      - name: ui
        image: guestbook:v1
```

Rollouts are drop-in

You can easily migrate from a `Deployment` to a `Rollout` by simply changing the `apiVersion` and `kind` fields.

The `Rollout` CRD

Understand the core resource in Argo Rollouts

deploy.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: guestbook-ui
spec:
  replicas: 5
  selector:
    matchLabels:
      app: guestbook
  template:
    metadata:
      labels:
        app: guestbook
    spec:
      containers:
      - name: ui
        image: guestbook:v1
```

rollout.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: guestbook-ui
spec:
  replicas: 5
  selector:
    matchLabels:
      app: guestbook
  template:
    metadata:
      labels:
        app: guestbook
    spec:
      containers:
      - name: ui
        image: guestbook:v1
```

Rollouts are drop-in

You can easily migrate from a `Deployment` to a `Rollout` by simply changing the `apiVersion` and `kind` fields.

`Rollouts` support most fields of a standard `Deployment`:

- ✅ It manages `ReplicaSets`
- ✅ It creates `Pods`
- ✅ It uses the same template field as `Deployments`

The `Rollout` CRD

Understand the core resource in Argo Rollouts

rollout.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: guestbook-ui
spec:
  replicas: 5
  selector: ...
  template: ...
  strategy:
    canary:
      steps:
      - setWeight: 20
      - pause: {}
      - setWeight: 50
      - pause: {duration: 10s}
```

rollout.yaml

```yaml
spec:
  strategy:
    blueGreen:
      activeService: guestbook-svc
      previewService: guestbook-preview-svc
      autoPromotionEnabled: false
```

Rollouts allow strategy fine-tuning

It expands on the `Recreate` and `RollingUpdate` strategies by adding support to `Canary` and `BlueGreen` strategies. The `Canary` strategy is very flexible, and can be customized to fit most projects' needs.

Argo Rollouts

Core Rollout Strategies

Core Rollout Strategies

Section overview

1 Blue-Green Deployment Strategy

1. Understanding the concepts behind Blue-Green Deployments, as well as its limitations.
2. Implementing a Rollout leveraging the Blue-Green Strategy.

2 Canary Deployment Strategy

1. Understanding the concepts behind Canary Deployments, as well as its limitations.
2. Implementing a Rollout leveraging the Canary Strategy.

3 Further Canary Patterns

Argo Rollouts

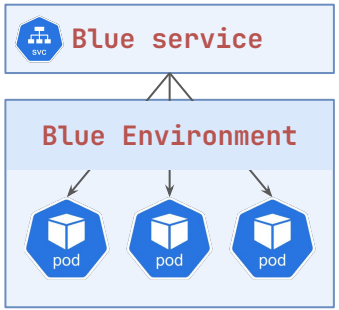
Blue-Green Deployments

Blue-Green Deployments

Understand the mechanics of Blue-Green deployments

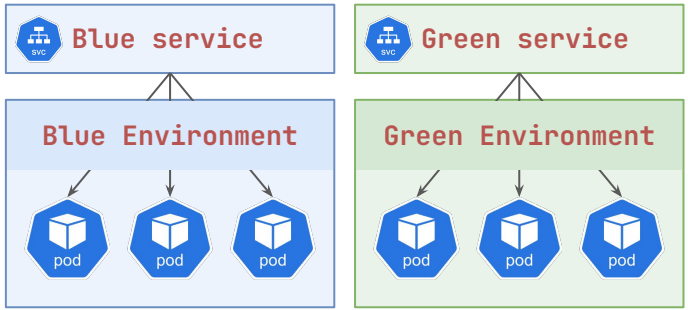
Stage 1: Only Blue Environment

> The application's stable version is running.



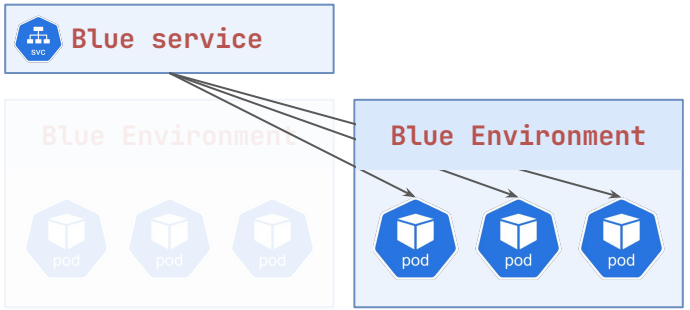
Stage 2: Blue & Green Environments Side by Side

> A new `ReplicaSet` is created as the Green Environment, and the Green Service points towards its pods.



Stage 3: Green Environment as the New Blue

> When the new Green Environment is deemed healthy, it's promoted as the new Blue Environment.



Argo Rollouts

Argo Rollouts

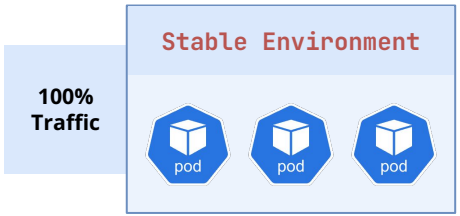
Canary Deployments

Canary Deployments

Understand the mechanics of Canary deployments

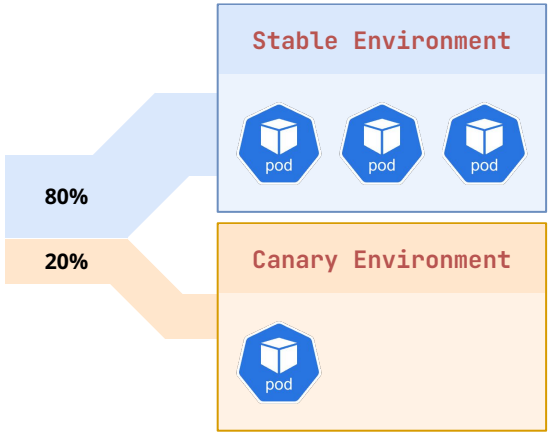
Stage 1: Only Stable Environment

The application's stable version is running.



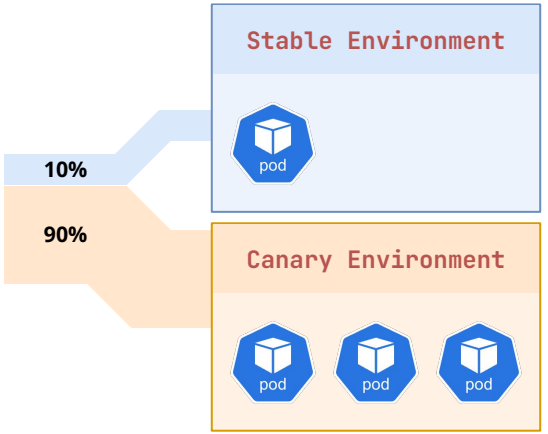
Stage 2: Start of Gradual Shift

A new Canary Environment is created and receives a portion of traffic.



Stage 3: Continuation of Gradual Shift

The traffic is gradually shifted to the Canary Environment until it receives all the traffic.



Argo Rollouts

Argo Rollouts

Advanced Traffic Management

Advanced Traffic Management

Section overview

1 Limitations of Replica-Weighted Strategies

1. Understanding the challenges of precise traffic splitting based solely on pod counts.
2. The need for more advanced traffic controls beyond standard Kubernetes services.

2 Gateway API & Environment Setup

1. Installing the Kubernetes Gateway API CRDs and deploying Traefik as the Gateway Controller.
2. Configuring the Argo Rollouts Gateway API Plugin to enable advanced traffic management.

3 Traffic-Weighted Canary Deployments

1. Decoupling traffic distribution from replica counts for granular control.
2. Exploring how Argo Rollouts dynamically manages `HTTPRoute` resources.

Advanced Traffic Management

Section overview

4 Header-Based Routing

1. Implementing deterministic routing for QA and testing purposes.
2. Combining traffic weights with specific route matching rules.

Argo Rollouts

Replica-Weighted Limitations

Replica-Weighted Limitations

Understand the limitations of replica-weighted Canary deployments

Limitation 1: The Precision Problem

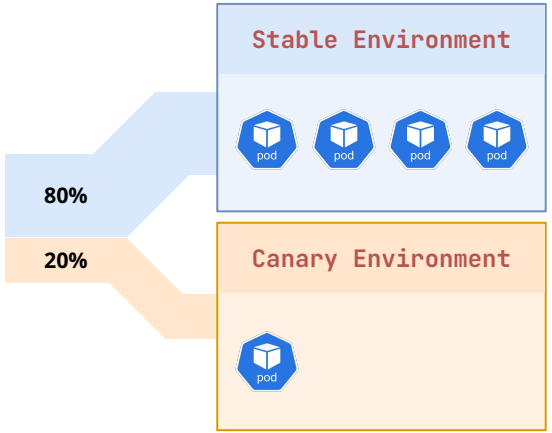
How do we achieve a 10% traffic split if our `ReplicaSet` has only **two** `Pods`?

Limitation 2: Resource Waste

To achieve fine-grained steps (1%, 2%, 5%), you are forced to over-provision your infrastructure.

Limitation 3: No Header-Based Routing

It's not possible to redirect traffic predictably with the use of custom headers.



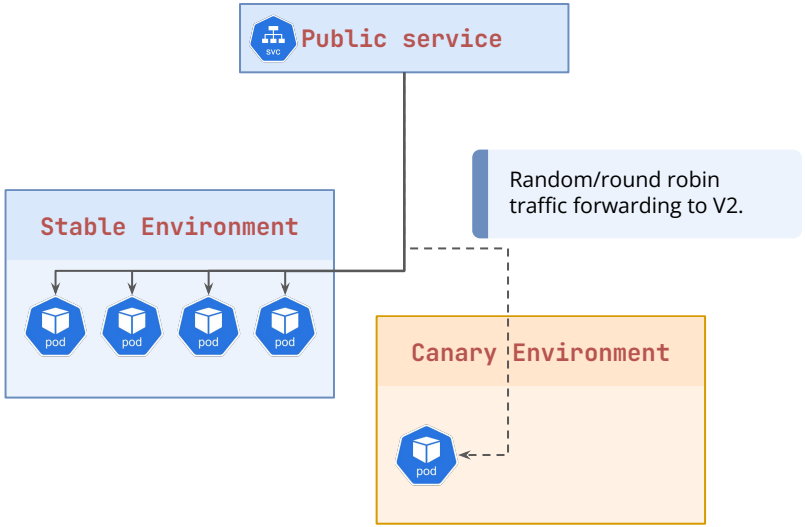
Argo Rollouts

Traffic-Weighted Canary

Traffic-Weighted Canary

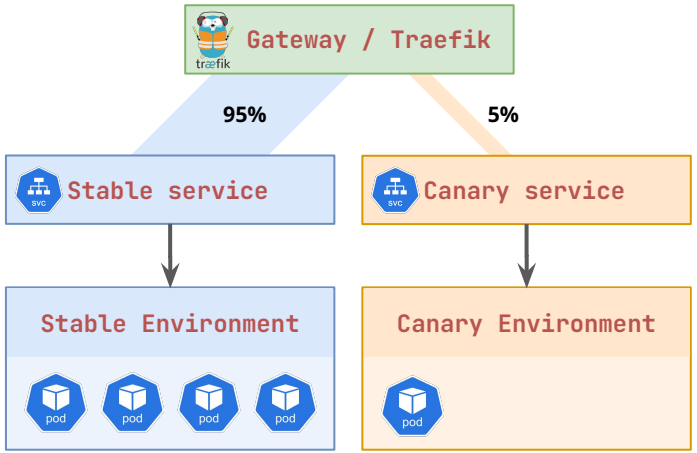
Understand how to achieve precise weighting with Rollouts and the Gateway API

Before: Replica-Weighted



After: Traffic-Weighted

The logic for weighting the traffic is done by the Gateway API (or Ingress) rather than the standard Kubernetes services.



Argo Rollouts

Header-Based Routing

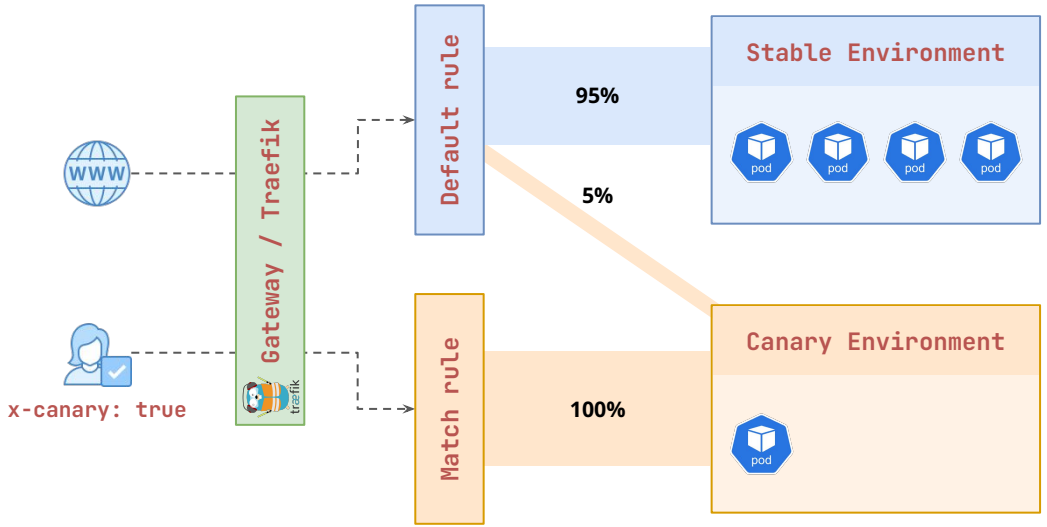
Header-Based Routing

Understand how to redirect traffic based on header values

Header-Based Routing allows us to fine-tune the redirection behavior based on request `headers`.

When the `header` matches the predefined value, it is redirected predictably according to the specified rule.

When the `header` does not match any predefined rules, it is redirected according to the default traffic split.



Header-Based Routing

Understand how to redirect traffic based on header values

header-based.yaml

```yaml
spec:
  strategy:
    canary:
      trafficRouting:
        managedRoutes:
          - "qa-override"
        # ... trafficRouting configuration ...
      steps:
      - setWeight: 1
      - setCanaryScale:
          weight: 20
      - setHeaderRoute:
          name: "qa-override"
          match:
            - headerName: "x-canary"
              headerValue: {exact: "true"}
      - pause: {}
      - setWeight: 20
      - pause: {duration: 30s}
```

Scales the Canary environment to have *some* capacity to serve requests.

Defines the specific header route that must match for traffic to be forwarded to the Canary environment.

Argo Rollouts' Gateway API plugin will pick up this change and properly adjust the HTTPRoute resource with a new, high priority rule.

Define the specific weights for the gradual promotion of public traffic.

Argo Rollouts

Automated Analysis and Promotion

Automated Analysis and Promotion

Section overview

1 Prometheus & Metrics Architecture

1. Overview of the monitoring stack and Service Discovery.
2. Installing Prometheus via the community Helm chart in the monitoring namespace.

2 Automated Analysis & Promotion

1. Integrating metrics into the rollout process to enable automated decision-making.
2. Defining Analysis Templates and implementing self-healing.

3 Analysis in Canary and Blue-Green Strategies

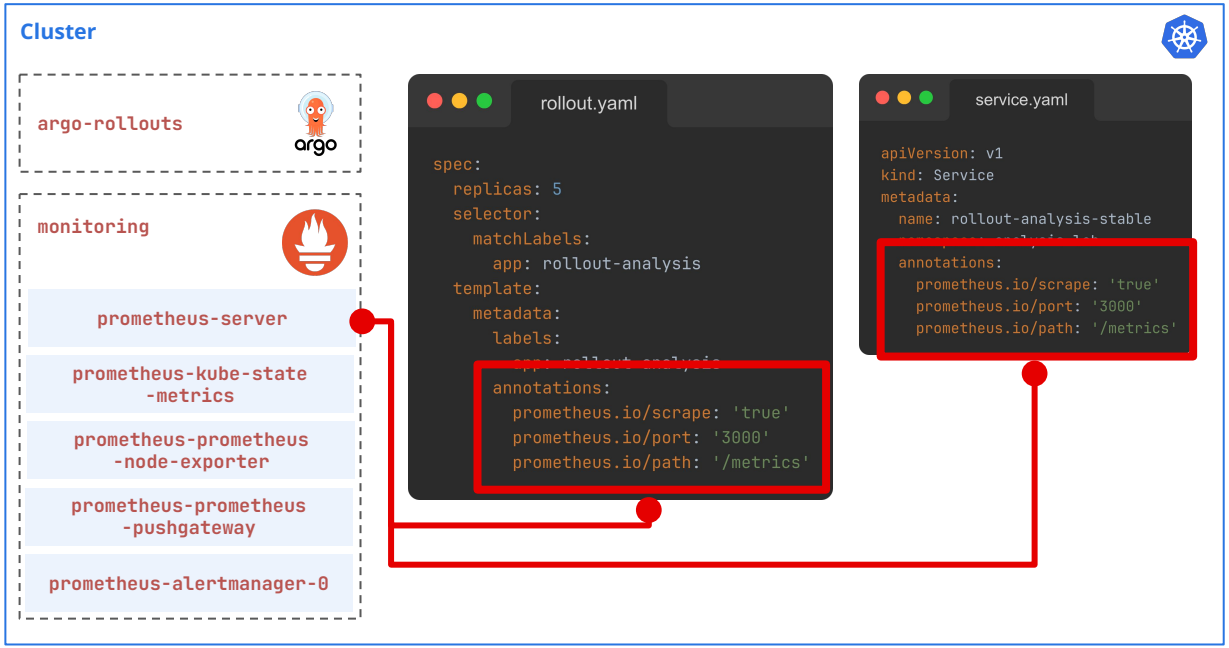
1. Canary Analysis: Running analysis as a step or continuously in the background.
2. Blue-Green Analysis: Validate Preview environments before switching traffic.
3. Handling arguments: Parameterizing queries and thresholds for reusability.

Argo Rollouts

Prometheus Architecture

Prometheus Architecture

A (very) high level overview of how Prometheus collects our metrics



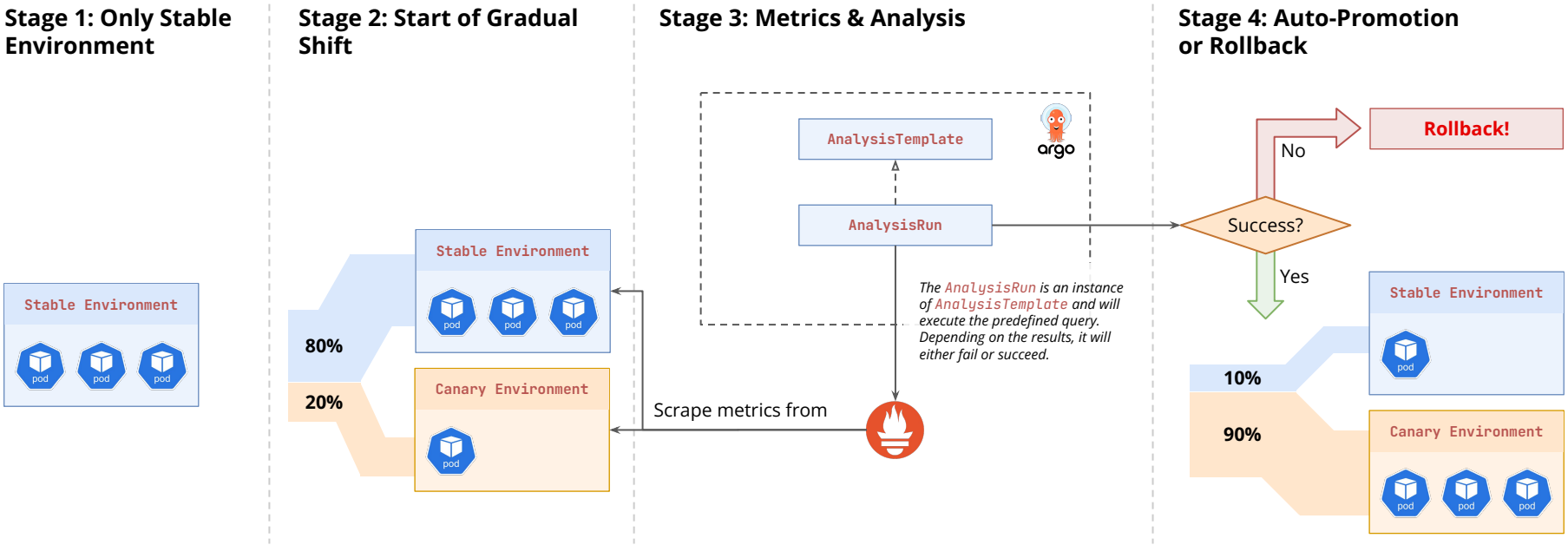
Argo Rollouts

Argo Rollouts

Automated Analysis

Automated Analysis

Understand how analyses can be used to automatically promote new revisions



Argo Rollouts